

OMNI-4DGS: A Unified Framework for Fast, Lightweight and Accurate Video-to-Model Reconstruction

Ali Emre Senel
Bocconi University
Milan, Italy

Tebe Nigrelli
Bocconi University
Milan, Italy

Stefana Chiriac
Bocconi University
Milan, Italy

Abstract

Dynamic scene reconstruction converts videos into compact, renderable 4D models. The dominant approach, Native 4D Gaussian Splatting is fast and effective but often suffers from Gaussian overgrowth, high VRAM use, large checkpoints, slow rendering, and fragile pruning or densification choices. OMNI-4DGS studies these quality-efficiency tradeoffs by jointly evaluating representation, rendering, and training decisions. We ablate covariance type, RGB versus 4DSH, rendering strategy, pruning schedules, ESS, dropout, and motion regularization across quality and efficiency metrics. Our best tested quality-compact preset improves visual quality while keeping model size compact, reaching 34.42 PSNR/29k Gaussians on bouncingballs and 31.89 PSNR/81k on trex, improving the quality-compactness tradeoff for practical 4DGS deployment.

github.com/AliEmreSenel/4DGS

1. Introduction

Gaussian Splatting has been used extensively for scene reconstruction from videos, thanks to its relatively small training requirements. In the 4D setting (4DGS), frames from a video are used to train a model that represents a scene in space and time. This representation can then be rendered from novel camera views as a function of time, position, orientation, and space.

In a 4DGS model, the scene is represented by many Gaussian primitives, parametrized by a position vector μ , which places their center in space, and a covariance matrix $\Sigma = \Sigma^T$ that allows anisotropic deformation. In practice, this covariance is parameterized through scale and rotation rather than by storing an arbitrary matrix directly. Color is associated with each Gaussian and encoded by basis coefficients such as Spherical Harmonics (SH) or, in Native 4DGS, 4D Spherindrical Harmonics (4DSH), which approximate view-dependent and time-dependent ra-

diance. Compared to constant color, these coefficients allow smoother appearance at the cost of more variables. Both anisotropic covariance and richer color bases can reduce the number of Gaussians needed by increasing per-primitive expressiveness.

Training iteratively reconstructs ground-truth images by rendering the Gaussians and optimizing image-similarity losses. The final output is a list of Gaussians, which can be projected to a camera plane and rasterized to obtain a reconstructed image. During inference, each pixel color is obtained by integrating Gaussian contributions along the view ray, while accounting for opacity, distance, and remaining transmittance.

Extending the task from static scenes to dynamic scenes, where objects move or change shape and color, Native 4D Gaussian Splatting (4DGS-Native) [1] adds a time coordinate and time covariance components to each Gaussian. Rendering must therefore first condition the Gaussians with respect to time before rendering them as in the 3D setting. All mentions of 4DGS refer to the native representation.

Although dynamic scene reconstruction has produced impressive results, many problems persist: scenes contain many low-importance Gaussians, which increase training time; rendering is limited by costly sorting algorithms; and initialization or training strategies are non-obvious. In this paper, we combine several improvements into a unified implementation, evaluating performance through ablations.

1.1. Gaussian Representation

To understand where these tradeoffs arise, we first describe the Gaussian representation itself. The parametrization of Gaussians in Native 4DGS is a tradeoff between using few variables and being expressive enough that fewer Gaussians are necessary to describe a scene. In 4DGS-Native, each Gaussian is characterized by a position vector $(\mu_x, \mu_y, \mu_z, \mu_t) = \mu \in \mathbb{R}^4$ representing the center of the Gaussian, that is, the mean of the distribution; scale parameters and a 4D rotation, represented using two quaternions

$r_1, r_2 \in \mathbb{H}$, which together define the covariance $\Sigma \in \mathbb{R}^{4 \times 4}$ and distort the Gaussian in time and space. For color, the native Gaussians are equipped with opacity $o \in [0, 1]$ and 4D Spherindrical Harmonic (4DSH) coefficients, which approximate view- and time-dependent color. In our ablations, the rows labeled 4DSH use spatial degree 3 with a temporal basis, i.e. $(m+1)^2(m_t+1)$ coefficients per color channel; for $m=3$ and $m_t=1$, this gives 32 coefficients per channel, or 96 RGB scalars per Gaussian. SH(0) corresponds to plain RGB colors, while the spatial-only 4DSH setting is an outside-the-paper control.

$$G_i = (\mu_i, S_i, (r_1, r_2), o_i, \overrightarrow{4DSH_i}) \quad (1)$$

Variations of this representation have also been developed to reduce the number of variables, which reduces training time and storage size but also lowers expressiveness. Isotropic Gaussians are spherical in the spatial dimensions and keep the temporal dimension independent, allowing them to shrink or grow over time while using fewer scalars. This is encoded with a 3×3 spatial covariance proportional to the identity matrix and a separate temporal variance. Here, $I_{3 \times 3}$ represents the 3×3 identity matrix.

$$\Sigma = \begin{pmatrix} \Sigma_{xyz} & \Sigma_{xyz,t} \\ \Sigma_{t,xyz} & \sigma_t \end{pmatrix} \text{ with } \Sigma_{xyz} = s_{xyz}^2 I_{3 \times 3} \quad (2)$$

$$\Sigma = \Sigma^T \Rightarrow \Sigma_{xyz,t} = \Sigma_{t,xyz}^T$$

1.2. Projection and Rendering

Images may be reproduced from 4DGS-Native Gaussians by first conditioning the distributions in time, which produces a 3D normal distribution. This colored “cloud” is mapped to a 2D camera plane using a world-to-camera projection matrix, reproducing a pixel image. During training, the operation enables the calculation of loss, since the reprojected image can be directly compared to one of the reference images. Generalization loss can also be calculated with images the model was not trained on. Image reconstruction is obtained by first fixing camera position and orientation, projecting the visible Gaussians to screen space, and alpha-blending their projected 2D footprints in depth order. The color of each pixel is obtained by accumulating per-Gaussian color contributions according to opacity and remaining transmittance. The formula for the final color is traditionally sort-dependent, but we also investigate sort-free rendering, as it was shown to reduce render time in related work [2].

Sort-based rendering consists of filtering the view to only consider relevant Gaussians, and processing them one at a time, integrating their colors in order, to obtain the resulting color of a pixel. For rendering, overall opacity is calculated sequentially to obtain Transmittance $T_i(p, t)$, that is, the “leftover” light level, which modulates the color con-

tribution $c_i(v, t)$ for each successive Gaussian. Remaining light T_{N+1} is attributed to background color c_{bg} . In the most general formulation, color is view-direction dependent, as well as time dependent. Moreover, the formula offers limited parallelization, as the sorting operation is a bottleneck.

$$\begin{cases} \alpha_i(p, t) = o_i G_i^{2D}(p | t) G_i^t(t) \\ T_i(p, t) = \prod_{j=1}^{i-1} (1 - \alpha_j(p, t)) \end{cases} \quad (3)$$

$$C_p(t, v) = \sum_{i=1}^N T_i(p, t) \alpha_i(p, t) c_i(v, t) + T_{N+1}(p, t) c_{bg} \quad (4)$$

Sort-free rendering, first proposed in [3], removes the depth-sorting step by computing color through an unordered weighted sum. We used [2] as our reference implementation because it combines sort-free rendering with compact, mobile-oriented 3DGS components, including MLP-predicted weights and opacity. In this formulation, transmittance is computed as an unsorted product, while weights w_i depend on viewing angle, camera position and distance. Since [2] is based on 3DGS, we extended the MLP architecture by concatenating three time features, $\Delta \frac{t}{T}$, t_{norm} , and $m_{t(t)}$, to the MLP inputs. For clarity, we provide the original formula from [2], for computing pixel color and Gaussian MLP weights. In the formulas, Δx_i is the screen-space offset between the pixel and the projected Gaussian center; Σ_i is the projected 2D covariance matrix of the Gaussian footprint; d_i is the Gaussian depth in camera coordinates; s_{max} is the maximum component of the Gaussian scale in camera coordinates; and s_i and r_i are the Gaussian scale and rotation parameters.

$$C_{pix} = (1 - T) \frac{\sum_{i=1}^N c_i \alpha_i w_i}{\sum_{i=1}^N \alpha_i w_i} + T c_{bg} \quad (5)$$

$$T = \prod_{j=1}^N (1 - \alpha_j) \quad (6)$$

$$\alpha_i = o_i \exp\left(-\frac{1}{2} \Delta x_i^T \Sigma_i^{-1} \Delta x_i\right) \quad (7)$$

$$w_i = \underbrace{\varphi_i^2}_{\text{MLP override}} + \underbrace{\frac{\varphi_i}{d_i^2}}_{\text{proximity effects}} + \underbrace{\exp\left(\frac{s_{max}}{d_i}\right)}_{\text{distance effects}} \quad (8)$$

$$P_i = \frac{\mu_i - k_v}{\|\mu_i - k_v\|} \text{ camera to Gaussian unit direction} \quad (9)$$

$$\begin{cases} \tau_i = (\Delta \frac{t}{T}, t_{\text{norm}}, m_{t(t)}) \\ F = \text{MLP}_f(P_i, s_i, r_i, \overrightarrow{4\text{DSH}_i}, \tau_i) \\ \varphi_i = \text{ReLU}(\text{MLP}_\varphi(F)) \\ o_i = \sigma(\text{MLP}_o(F)) \end{cases} \quad (10)$$

Other optimizations have also been developed for the rendering (inference) operation: following [2], contribution-based pruning removes Gaussians using accumulated importance votes based on opacity and scale statistics. Visibility masks are also an option for selectively loading Gaussians at render time: [4] proposes key-frame temporal filtering with binary labels at sparse key frames, so the rendering step loads only Gaussians visible in the nearby temporal window. Both techniques result in faster inference, by drastically reducing memory loading and problem size.

1.3. Training

Backpropagation is used for training, minimizing image-similarity metrics, with additional loss components as they are introduced by the architecture. We use the Adam optimizer and batch training to improve stability [1].

As memory usage is proportional to the number of Gaussians, one seeks to minimize this number by pruning less relevant ones. Opacity pruning drops Gaussians with opacity values below a chosen threshold [1]. Contribution pruning accumulates a contribution value over training steps and drops Gaussians that remain insufficiently relevant for enough iterations [2]. Spatio-Temporal pruning ranks Gaussians by visibility and temporal persistence [4]; in our D-NeRF ablations it removes 15% per pass, applied repeatedly between iterations 1500 and 11250 at interval 750, while the `_prune` variants use 85%. Finally, Grid pruning deduplicates Gaussians by position, velocity, temporal scale, and time placement [5]. Since most pruning rules tend to be correlated, in trying to achieve the same goal, we implement Spatio-Temporal pruning, and keep opacity and size-based pruning from 4DGS-Native. Related to pruning, we apply densification to increase the number of Gaussians where necessary. We also implement Edge-Guided densification, adding Gaussians near image edges [6], and Gradient-Based densification, splitting Gaussians with high loss gradients [7].

Rasterization is performed through custom CUDA kernels for speed. However, compatibility issues arise because the codebases use different versions. For this reason, we opted for a mixed pruning-densification schedule, aiming to simulate a better-than-random initialization, as opposed to a point cloud initialization from an existing model [5].

1.4. Loss

The core training objective is photometric fidelity, while additional regularizers are optional and depend on the archi-

ture. The reconstruction loss is expressed as a weighted combination of the pixel-wise L1 distance between the original and reconstructed images and an SSIM-based structural term, which accounts for luminance, contrast, and texture:

$$\mathcal{L}_{\text{rgb}} = (1 - \lambda_{\text{dssim}})\mathcal{L}_1 + \lambda_{\text{dssim}}(1 - \text{SSIM}) \quad (11)$$

For dynamic outdoor scenes, photometric supervision alone is not sufficient: transparent sky regions may still be filled with foreground Gaussians. In our code, the optional sky-mask term is implemented as a ground-truth-alpha opacity loss on sky pixels:

$$\mathcal{L}_{\text{sky}} = -\frac{1}{|\Omega|} \sum_{p \in \Omega} m_{\text{sky}}(p) \log(1 - \alpha(p)) \quad (12)$$

Dynamic Gaussians also require temporal regularization to avoid physically implausible motion. The rigidity loss encourages nearby Gaussians to move with similar velocities. It is weighted by spatial distance and averaged over $k = 20$ neighbours:

$$\mathcal{L}_{\text{rigid}} = \frac{1}{k \cdot G} \sum_{i=1}^G \sum_{j \in \mathcal{N}(i)} e^{-100 \cdot d(i,j)} \cdot \|\dot{\mu}_i - \dot{\mu}_j\|_2 \quad (13)$$

A global motion loss further suppresses high-velocity Gaussians that are unlikely to correspond to real scene motion:

$$\mathcal{L}_{\text{motion}} = \frac{1}{G} \sum_{i=1}^G \|\dot{\mu}_i\|_2 \quad (14)$$

where $\dot{\mu}_i$ is the temporal velocity estimated by finite difference. Only $\mathcal{L}_{\text{rgb}}$ is active in the `trex` and `bouncingballs` ablations; sky, rigidity, motion, and depth losses are described for completeness, but are disabled in those configs.

Reconstruction quality is then reported using three complementary image metrics:

- Peak Signal-to-Noise Ratio (PSNR): measures pixel-level fidelity.
- Structural Similarity Index Measure (SSIM): captures structural similarity in luminance, contrast, and texture.
- Learned Perceptual Image Patch Similarity (LPIPS): estimates similarity from neural feature activations.

1.5. Uncertainty-Aware Loss

When optimization is applied equally to all Gaussians, low-quality Gaussians remain unconstrained, producing visual inconsistencies. Uncertainty Awareness addresses this problem, activating after base-model convergence [8]. First, uncertainty $u_{i,t}$ is estimated per Gaussian per frame from alpha-blending weights: well-observed Gaussians contribute strongly to many pixels, thus receiving low uncertainty:

$$\sigma_{i,t}^2 = \frac{1}{\sum_{h \in P_{i,t}} (T_{i,t}^h \alpha_i)^2}, \quad u_{i,t} = \begin{cases} \sigma_{i,t}^2 & \mathbb{I}_{i,t} \\ \varphi & \end{cases} \quad (15)$$

with $\varphi = 10^6$. The term $\mathbb{I}_{i,t}$ detects Gaussian convergence in color, which forces $u = \varphi$ unless every pixel in the Gaussian footprint has color residual below $\eta_c = 0.5$. This mechanism prevents unconverged Gaussians from being attributed high certainty. Next, the top 2% highest-confidence Gaussians over a significant period (≥ 5 frames) become key nodes V_k . From the point cloud of key nodes, kNN is used to add edge connections between key nodes. Each non-key Gaussian is assigned to the closest key node over the full sequence.

Key-node loss ensures that reliable Gaussians do not drift from their well-trained and certain positions, while their neighbourhoods move consistently. This is enforced by anchoring them to their pretrained positions μ° and applying motion locality constraints:

$$\mathcal{L}_{\text{key}} = \sum_t \sum_{i \in V_k} \|\mu_{i,t} - \mu_{i,t}^\circ\|_{U_{w,t,i}^{-1}}^2 + \mathcal{L}_{\text{motion,key}} \quad (16)$$

The uncertainty matrix $U_{i,t} = R_{\text{wc}} \text{diag}(u, u, 100u) R_{\text{wc}}^T$ encodes the special treatment of the depth direction under monocular depth unreliability by downweighting depth deviations in the Mahalanobis loss. $\mathcal{L}_{\text{motion,key}}$ combines isometry, rigidity, rotation, velocity and acceleration constraints ($\lambda_{\text{iso}} = \lambda_{\text{rigid}} = 1.0$, $\lambda_{\text{rot}} = \lambda_{\text{vel}} = \lambda_{\text{acc}} = 0.01$).

Non-key node loss pulls uncertain Gaussians toward positions predicted by interpolating the motion of their key neighbours. It uses Dual Quaternion Blending (DQB) to interpolate multiple rotations:

$$\mathcal{L}_{\text{non-key}} = \mathcal{L}_{\text{pretr,non-key}} + \mathcal{L}_{\text{DQB,non-key}} + \mathcal{L}_{\text{mot,non-key}} \quad (17)$$

$$\mathcal{L}_{\text{pretr,non-key}} = \sum_t \sum_{i \notin V_k} \|\mu_{i,t} - \mu_{i,t}^\circ\|_{U_{w,t,i}^{-1}}^2 \quad (18)$$

$$\mathcal{L}_{\text{DQB,non-key}} = \sum_t \sum_{i \notin V_k} \|\mu_{i,t} - \mu_{i,t}^{\text{DQB}}\|_{U_{w,t,i}^{-1}}^2 \quad (19)$$

DQB is a soft interpolation for rotations, where $\mu_{i,t}$ remains free and may deviate when the photometric loss provides a stronger signal, preserving non-rigid deformation. Density control is disabled in the first 10% and last 20% of USPLAT iterations to maintain optimization stability.

1.6. Gaussian Regularization

Taking inspiration from [6], we implement **Random Dropout Regularization**, extending it from 3D to 4D, to ensure that remaining Gaussians are not overly dependent on any particular subset of primitives. At each training iteration, a random mask disables a fraction of Gaussians before rendering, while the full model remains the reference target. This encourages neighboring primitives to share responsi-

			existing	heavily modified	re-implemented	4DGS-Nat.	1000FPS	Instant4D	MobileGS	Usplat4D	Omni-4DGS
Gaussians	Gaussians	4D				■	■	■		■	■
		3D							■		
	Rotation	Quaternion				■	■	■		■	■
		Rotation Matrix							■		
	Shape	Isotropic						■			■
		Anisotropic				■	■		■	■	■
	Color Basis	RGB						■			■
		SH(1)							■		■
		SH(3)				■	■		■	■	■
Init	Point Cloud	Random				■	■				■
		MegaSAM						■			
Train	Weighting	Uncertainty								■	■
	Sampling	Batch in Time				■	■				■
	Grid Reliance	Voxelization						■	■	■	■
Prune	Criterion	Contribution				■			■		■
		Gradient Loss				■					■
	Quantile Filter	Spatio-Temporal						■	■		■
		Opacity						■			■
	Strategy	One-shot					■	■		■	■
		Scheduled									■
	Increase	Gradient Based				■					■
		Edge-guided Split									■
Render	Dropout	Dropout									■
	Loading	Visibility Mask					■				■
Render	Raster	Sort-based				■	■	■		■	■
		Sort-free							■		■

Table 1. Contributions of Each Architecture

bility for explaining observed pixels, reducing sparse-view overfitting, floating artifacts, and hollow artifacts. During inference, all Gaussians are restored, effectively aggregating many low-complexity submodels. The intended effect is smoother geometry and improved generalization, while later refinement can recover high-frequency details.

We also implement an approximate **Edge-guided Splitting Strategy** (ESS) inspired by [6]. ESS increases Gaussian density near image discontinuities instead of distributing new primitives uniformly. Our implementation is not a faithful DropoutGS port: it uses Sobel edges on the ground-truth image rather than a rendering-error map, and it performs a periodic mask-split pass rather than integrat-

ing edge probabilities into the standard densification rule. This complements dropout: dropout regularizes the existing primitives, while ESS determines where extra primitives should be allocated.

2. OMNI Architecture

Having described the individual components, we now explain how they are integrated into our unified OMNI-4DGS pipeline. We combine components from different codebases into a single unified architecture to test the effectiveness of each component. We summarize our codebase in Table 1, which also highlights the efforts required to combine them.

We train ablations on the `trex` dataset with inference of 400×400 , allowing results to be compared with SOTA [1]. We record PSNR, SSIM, LPIPS, peak RAM and VRAM usage, memory footprint, number of Gaussians, and render-time FPS. For completeness, we tested the `bouncingballs` dataset. We also run USPLAT ablations on limited 7k-step runs, due to the massive time overhead introduced by our reference USPLAT implementation (Appendix Section 3, while Section 2 explains Ablations Coverage). We run each ablation once, reporting the best checkpoint over training rather than necessarily the final 30k-step checkpoint. Although runs were allowed to continue up to 30k iterations, performance generally peaked earlier, at around 10k–15k iterations, after which mild degradation started. Each ablation was run once, computing Standardized Marginal Differences (SMD) to identify notable changes. However, these coefficients should be interpreted descriptively rather than as estimates of run-to-run statistical significance. Baseline values are taken from the corresponding papers rather than rerun under identical hardware. Moreover, FPS depends strongly on resources and renderers, so comparisons are indicative rather than definitive.

3. Ablation Experiments

We use this unified implementation to isolate the effect of each design choice. From analyzing the `trex` dataset, component effects are not independent: initialization, Gaussian parametrization, color representation, pruning, and rendering all interact through the same budget of Gaussians, parameters, and execution time. To compare effects across metrics with different units, we report relative changes and direction-corrected standardized marginal differences across non-target axes, where positive indicates improvement and negative indicates degradation. Since each ablation was run once, these values are descriptive rankings rather than estimates of run-to-run statistical significance.

3.1. Visual Quality

Visual quality improves most from anisotropic Gaussians and 4DSH color, with the best pruning schedule stemming from interleaved pruning and densification. Anisotropic outperforms isotropic with a higher PSNR (31.78 vs. 29.66, +7.2%, SMD = 1.42), higher SSIM (0.9797 vs. 0.9713, +0.9%, SMD = 1.63), and lower LPIPS (0.0218 vs. 0.0416, -47.6%, SMD = 2.55), showing that directional covariance is important for representing geometry and motion-dependent appearance. 4DSH also strongly improves quality over RGB, raising PSNR from 29.81 to 31.63, raising SSIM from 0.9729 to 0.9780, and reducing LPIPS from 0.0349 to 0.0285. Interleaved pruning and densification gives the best mean PSNR, SSIM, and LPIPS among pruning schedules, while early initialization pruning is not competitive with the strongest quality-compact configurations. Pruning is therefore most useful when paired with densification and evaluated jointly with the Gaussian representation.

3.2. Memory Usage

This quality gain, however, comes with a memory cost. Memory usage is mainly determined by color representation, pruning strategy, and Gaussian count. RGB is the most memory-efficient color model, reducing checkpoint size from 45.1 to 4.8 MB (-89.3%, SMD = 2.78) and evaluation VRAM from 784.8 to 631.9 MB (-19.5%, SMD = 0.64). However, this comes with a large quality loss, so RGB is best suited to compact deployment rather than high-fidelity reconstruction. 4DSH gives much better quality, but increases memory. Pruning is beneficial when it reduces active primitives during training: interleaved pruning and densification reduces evaluation VRAM from 752.8 to 575.0 MB (-23.6%, SMD = 0.75) and gives the smallest mean checkpoint size among pruning schedules. No pruning is less compact than the interleaved schedule, but remains competitive only when speed is prioritized over memory.

3.3. Gaussian Count

The same trend is visible when looking at Gaussian count. Gaussian count is highly scene-dependent, but it qualitatively measures how efficiently the Gaussians are used. Interleaved pruning and densification gives the lowest mean count among pruning schedules, lowering the average to 72.8k Gaussians while also giving the lowest VRAM and checkpoint size. Anisotropic Gaussians also reduce the required primitives from 82.6k to 70.8k while improving quality, meaning they increase representational efficiency rather than merely adding complexity. 4DSH and no-dropout also reduce counts, likely because better appearance modeling and more stable optimization reduce densification pressure. In contrast, ESS increases the mean

count from 71.2k to 82.2k, so its edge allocation must be balanced against compactness.

3.4. Inference FPS

Speed shows a slightly different pattern. Inference FPS benefits most from removing dropout and from reducing active primitive cost. No-dropout improves FPS from 493.7 to 791.4 (+60.3%, SMD = 0.69), likely because it produces fewer Gaussians and a simpler final model. Final pruning gives the highest mean FPS among pruning schedules at 690.4 FPS, while interleaved pruning reduces count and VRAM but reaches 621.7 FPS on average. This suggests that throughput depends on primitive distribution, memory layout, and renderer implementation, not only the total number of Gaussians.

3.5. Training Time

Training time is most affected by dropout, pruning, and color representation. Removing dropout gives the largest improvement, reducing training time from 89 to 36 minutes (−60.3%, SMD = 2.01). Dropout adds optimization cost without clear quality or compactness gains. Interleaved pruning and densification remains attractive for VRAM, serialized model size, and quality, but it increases training time relative to the no-pruning schedule because pruning and densification add control overhead. RGB trains faster than 4DSH, but the quality loss makes it less suitable for high-fidelity reconstruction. Dropout is the most harmful choice for training efficiency because it slows convergence.

3.6. Role of each Component

These results suggest that no component should be evaluated in isolation. Anisotropic Gaussians are one of the most beneficial components: they improve PSNR, SSIM, LPIPS, and reduce Gaussian count. Isotropic Gaussians use fewer variables per primitive but lose too much representational power. RGB reduces memory and checkpoint size but hurts quality, while 4DSH improves visual fidelity and reduces primitive count at the cost of higher VRAM and checkpoint size. Interleaved pruning best balances quality, count, memory, and VRAM; final pruning is strongest for FPS; early initialization pruning is not selected by the strongest quality-compact configurations. Dropout gives no clear benefit, and removing it improves time, FPS, count, and mean PSNR. Sort-free rendering was not tested as extensively as the other components because pilot runs were 2 – 3 × slower than the sorted renderer, while also giving worse overall performance. ESS and USPLAT likewise provided limited gains or excessive overhead in this setting.

We did not identify a clear winner among the ablations, beyond individual tendencies: the best combination must be considered on the basis of priorities. However, in our experiments the best quality-compact configuration

improves visual quality over the reported 4DGS-Native [1] and 4DGS-1K [4] baselines, while remaining more compact than 4DGS-Native but slower than 4DGS-1K:

aniso · 4DSH · sort · ESS · interleaved prune · no dropout

	Method	PSNR↑	SSIM↑	LPIPS↓	FPS↑	MB↓	Gauss↓
BBalls	4DGS	33.35	0.982	0.025	462	84	134k
	4DGS-1K	33.45	0.983	0.025	1509	13	20k
	Ours	34.42	0.985	0.016	460	18	29k
TRex	4DGS	29.85	0.980	0.019	202	792	1265k
	4DGS-1K	30.47	0.981	0.018	1361	118	189k
	Ours	31.89	0.984	0.016	386	50	81k

FPS is highly dependent on renderer and hardware; numbers are not comparable.

3.7. Discussion and Limitations

These results highlight the central theme of our study: 4DGS performance is governed by coupled tradeoffs rather than independent improvements. 4DGS design is a coupled resource tradeoff: each component affects Gaussian count, per-primitive parameters, memory bandwidth, CUDA time, and stability. Anisotropic covariance improves PSNR, SSIM, and LPIPS despite higher per-Gaussian cost by reducing required primitives. 4DSH similarly improves appearance and lowers densification pressure; on the other hand, RGB is insufficient for high fidelity. Pruning must follow the target deployment constraint: interleaved pruning balances quality, serialized compactness, and VRAM, while final pruning aids FPS. Best quality-compact results use anisotropy, 4DSH, sort rendering, ESS, interleaved pruning, and no dropout. Limitations include narrow datasets, noisy MOG capture, approximate USPLAT comparison, baselines taken from papers, hardware-dependent FPS, CUDA adaptations, and weak initialization causing floating Gaussians.

4. Conclusion

We presented **OMNI-4DGS**, a unified framework for fast, compact dynamic reconstruction built on Native 4D Gaussian Splatting, combining architectural choices from recent work into a single ablation study. Efficient 4DGS requires increasing expressiveness per Gaussian while controlling primitive growth: anisotropic covariance and 4DSH improve reconstruction quality; interleaved pruning and densification reduce Gaussian count, serialized model size, and VRAM, and a final one-shot prune further improves FPS at deployment. Dropout and the adapted sort-free renderer were not consistently beneficial across scenes. The **aniso · 4DSH · sort · ESS · interleaved prune · no dropout** configuration achieved the best tested quality-compactness tradeoff on both **tr**ex and **bouncingballs**.

References

- [1] Zeyu Yang *et al.*, 4D Gaussian Splatting: Modeling Dynamic Scenes with Native 4D Primitives, *arXiv preprint arXiv:2412.20720*, 2024, doi: [10.48550/arXiv.2412.20720](https://doi.org/10.48550/arXiv.2412.20720).
- [2] Xiaobiao Du, Yida Wang, Kun Zhan, and Xin Yu, Mobile-GS: Real-time Gaussian Splatting for Mobile Devices, *arXiv preprint arXiv:2603.11531*, 2026, doi: [10.48550/arXiv.2603.11531](https://doi.org/10.48550/arXiv.2603.11531).
- [3] Qiqi Hou *et al.*, Sort-free Gaussian Splatting via Weighted Sum Rendering, *arXiv preprint arXiv:2410.18931*, Oct. 2024, doi: [10.48550/arXiv.2410.18931](https://doi.org/10.48550/arXiv.2410.18931).
- [4] Yuheng Yuan, Qiuhong Shen, Xingyi Yang, and Xinchao Wang, 1000+ FPS 4D Gaussian Splatting for Dynamic Scene Rendering, *arXiv preprint arXiv:2503.16422*, 2025, doi: [10.48550/arXiv.2503.16422](https://doi.org/10.48550/arXiv.2503.16422).
- [5] Zhanpeng Luo, Haoxi Ran, and Li Lu, Instant4D: 4D Gaussian Splatting in Minutes, *arXiv preprint arXiv:2510.01119*, 2025, doi: [10.48550/arXiv.2510.01119](https://doi.org/10.48550/arXiv.2510.01119).
- [6] Yexing Xu, Longguang Wang, Minglin Chen, Sheng Ao, Li Li, and Yulan Guo, DropoutGS: Dropping Out Gaussians for Better Sparse-view Rendering, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pages 701–710.
- [7] Shuo Sun, Malcolm Mielle, Achim J. Lilienthal, and Martin Magnusson, High-Fidelity SLAM Using Gaussian Splatting with Rendering-Guided Densification and Regularized Optimization, *arXiv preprint arXiv:2403.12535*, 2024.
- [8] Yun Chien, usplat4d: Self-implemented USplat4D. 2026.

A Hardware Resources

Experiments were run on shared compute resources, so render-time FPS should be interpreted as hardware-dependent rather than as a fully hardware-normalized benchmark. Available resources included:

- 4x Lenovo ThinkSystem SR630 v4: 2x Intel Xeon 6517P 16-core CPUs, 1024 GB DDR5-6400M RAM, 2x NVIDIA H100 GPUs with 94 GB RAM.
- 4x Dell PowerEdge R650: 2x Intel Xeon 4316 20-core CPUs, 512 GB DDR4-3200 RAM.

B Ablation Cover

Each trained ablation was run once; reported effect sizes are descriptive rather than estimates of run-to-run significance. FPS comparisons are indicative because throughput depends on hardware resources and renderer settings.

Ablation axis	Variants tested	Dataset	Budget	Runs	Coverage / caveat
Covariance	Isotropic, anisotropic	trex	up to 30k	1 each	best checkpoint reported.
Color	RGB, SH(3)	trex	up to 30k	1 each	evaluates quality-memory tradeoff.
Pruning	None, early, final, inter-leaved	trex	up to 30k	1 each	schedules interact with den-sification.
Dropout	Enabled, disabled	trex	up to 30k	1 each	no repeated seeds.
ESS	Disabled, enabled	trex	up to 30k	1 each	Coupled to Gaussian count.
Rendering	Sort, adapted sort-free	trex	limited	1 each	Partial ablation; sort-free was much slower and not explored as deeply.
USPLAT / motion	Disabled, enabled	trex	7k	1 each	Limited ablation due to im-plementation overhead.
Dataset check	Best preset only	bouncingballs	up to 30k	1	Sanity check, not a full abla-tion sweep.
Baselines	4DGS, 4DGS-1K	trex, bouncingballs	reported	-	Taken from papers, not rerun under identical hardware.

Table 2.Scope of ablation coverage.

C Ablation Results

- MB size column includes optimizer checkpoint state size. True size is in the order of 1/3 of the MB size.
- FPS is a relative measure as it is hardware-dependent.
- **Iter** reports the checkpoint iteration, in thousands, selected as the best checkpoint for that row.

In the following tables, the best cells for each metric are colored in green, while the worst 25% of values are in red.

ABLATIONS					VISUAL								
Isotropy	Color	Prune	Drop	ESS	Iter $\times 10^3$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	VRAM $\downarrow$	MB $\downarrow$	Gauss $\times 10^3 \downarrow$	Train m $\downarrow$
aniso	RGB	early	no	no	11	30.61	0.971	0.033	889	624	4.5	54.5	34m
aniso	RGB	early	no	yes	8	30.51	0.972	0.036	516	683	3.2	38.8	26m
aniso	RGB	early	yes	no	11	29.82	0.972	0.031	400	544	4.7	57.8	51m
aniso	RGB	early	yes	yes	12	29.87	0.970	0.033	395	579	6.1	74.6	62m
aniso	RGB	final	no	no	11	31.04	0.967	0.037	662	666	4.6	55.9	39m
aniso	RGB	final	no	yes	8	30.53	0.966	0.038	757	718	3.3	39.7	27m
aniso	RGB	final	yes	no	9	29.94	0.967	0.030	522	558	3.7	44.7	47m
aniso	RGB	final	yes	yes	11	29.70	0.965	0.035	162	684	5.8	70.7	58m
aniso	RGB	none	no	no	8	30.87	0.969	0.036	343	614	3.0	36.3	26m
aniso	RGB	none	no	yes	12	30.55	0.964	0.038	406	672	5.9	72.3	42m
aniso	RGB	none	yes	no	9	29.77	0.966	0.032	789	558	3.7	45.1	46m

ABLATIONS					VISUAL								
Isotropy	Color	Prune	Drop	ESS	Iter $\times 10^3$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	VRAM $\downarrow$	MB $\downarrow$	Gauss $\times 10^3 \downarrow$	Train m $\downarrow$
aniso	RGB	none	yes	yes	9	29.61	0.965	0.032	600	579	4.1	50.4	47m
aniso	RGB	prune+dense	no	no	9	31.41	0.974	0.028	2233	608	2.4	29.0	26m
aniso	RGB	prune+dense	no	yes	14	31.38	0.973	0.031	369	656	4.1	50.6	45m
aniso	RGB	prune+dense	yes	no	11	30.34	0.974	0.029	411	374	2.8	34.0	63m
aniso	RGB	prune+dense	yes	yes	12	30.23	0.973	0.030	560	398	4.0	48.3	65m
aniso	4DSH	early	no	no	11	33.98	0.984	0.022	1098	675	24.4	39.3	37m
aniso	4DSH	early	no	yes	10	33.70	0.983	0.023	626	741	26.0	41.9	36m
aniso	4DSH	early	yes	no	12	33.88	0.984	0.032	190	827	31.5	50.8	69m
aniso	4DSH	early	yes	yes	10	33.77	0.984	0.018	444	653	31.3	50.5	62m
aniso	4DSH	final	no	no	11	34.05	0.980	0.021	818	676	24.8	40.0	35m
aniso	4DSH	final	no	yes	27	34.11	0.983	0.017	1196	698	15.5	25.1	97m
aniso	4DSH	final	yes	no	30	33.90	0.983	0.018	132	773	15.8	25.4	156m
aniso	4DSH	final	yes	yes	30	33.86	0.983	0.016	669	761	20.3	32.7	167m
aniso	4DSH	none	no	no	10	34.08	0.979	0.021	544	670	23.0	37.1	32m
aniso	4DSH	none	no	yes	10	33.72	0.979	0.022	682	733	27.1	43.7	34m
aniso	4DSH	none	yes	no	11	33.93	0.979	0.017	416	608	30.6	49.4	55m
aniso	4DSH	none	yes	yes	10	33.82	0.979	0.018	470	618	32.5	52.4	53m
aniso	4DSH	prune+dense	no	no	12	34.51	0.986	0.015	390	597	17.2	27.7	33m
aniso	4DSH	prune+dense	no	yes	11	34.42	0.985	0.016	460	621	18.1	29.2	35m
aniso	4DSH	prune+dense	yes	no	15	34.46	0.986	0.016	417	374	24.4	39.3	83m
aniso	4DSH	prune+dense	yes	yes	12	34.52	0.986	0.027	429	403	25.0	40.4	62m
iso	RGB	early	no	no	6	29.97	0.971	0.057	2468	755	1.3	28.5	19m
iso	RGB	early	no	yes	8	30.12	0.969	0.052	1222	658	2.0	45.2	26m
iso	RGB	early	yes	no	8	29.74	0.972	0.049	503	553	2.1	47.6	41m
iso	RGB	early	yes	yes	11	29.68	0.970	0.047	388	576	3.0	68.8	63m
iso	RGB	final	no	no	6	29.96	0.964	0.057	2403	712	1.3	29.9	18m
iso	RGB	final	no	yes	6	29.92	0.964	0.058	1384	661	1.4	31.3	20m
iso	RGB	final	yes	no	9	30.23	0.967	0.046	464	538	2.5	56.2	48m
iso	RGB	final	yes	yes	11	29.51	0.967	0.048	395	571	3.1	71.6	64m
iso	RGB	none	no	no	12	29.92	0.962	0.054	605	666	3.0	68.0	43m
iso	RGB	none	no	yes	10	28.96	0.965	0.053	1680	1581	2.6	59.1	4m
iso	RGB	none	yes	no	6	29.46	0.966	0.056	448	535	1.4	32.6	27m
iso	RGB	none	yes	yes	12	29.75	0.965	0.047	452	572	3.4	78.3	62m
iso	RGB	prune+dense	no	no	6	30.02	0.969	0.058	1878	602	1.0	23.5	18m
iso	RGB	prune+dense	no	yes	9	30.14	0.967	0.056	2509	665	1.6	36.4	29m
iso	RGB	prune+dense	yes	no	24	29.89	0.968	0.055	560	345	2.4	54.1	114m
iso	RGB	prune+dense	yes	yes	9	30.10	0.971	0.058	578	370	1.8	40.6	45m
iso	4DSH	early	no	no	8	31.99	0.974	0.046	626	673	22.9	39.4	24m
iso	4DSH	early	no	yes	19	32.07	0.973	0.043	273	802	46.2	79.5	63m
iso	4DSH	early	yes	no	11	32.23	0.976	0.040	547	636	33.1	56.9	53m
iso	4DSH	early	yes	yes	11	32.21	0.976	0.040	429	653	33.6	57.7	60m
iso	4DSH	final	no	no	12	31.98	0.970	0.045	699	701	33.1	56.9	40m
iso	4DSH	final	no	yes	17	31.91	0.968	0.046	817	809	44.9	77.3	54m
iso	4DSH	final	yes	no	12	32.04	0.972	0.040	453	639	35.9	61.7	66m
iso	4DSH	final	yes	yes	17	32.06	0.970	0.041	525	704	44.8	77.1	94m
iso	4DSH	none	no	no	8	31.90	0.970	0.047	381	673	23.8	40.8	27m
iso	4DSH	none	no	yes	19	32.00	0.968	0.046	466	820	47.5	81.7	67m
iso	4DSH	none	yes	no	11	31.87	0.971	0.041	552	912	34.1	58.7	57m
iso	4DSH	none	yes	yes	15	31.99	0.970	0.042	421	687	41.1	70.6	85m
iso	4DSH	prune+dense	no	no	9	31.95	0.974	0.045	412	584	17.9	30.8	26m
iso	4DSH	prune+dense	no	yes	9	31.99	0.974	0.045	341	617	18.1	31.1	25m
iso	4DSH	prune+dense	yes	no	15	31.99	0.975	0.042	565	313	24.2	41.6	74m
iso	4DSH	prune+dense	yes	yes	10	31.92	0.975	0.045	578	308	21.5	36.9	52m

Table 3.bouncingballs

ABLATIONS					VISUAL								
Isotropy	Color	Prune	Drop	ESS	Iter $\times 10^3$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	VRAM $\downarrow$	MB $\downarrow$	Gauss $\times 10^3 \downarrow$	Train m $\downarrow$
aniso	RGB	early	no	no	8	30.88	0.982	0.018	336	639	6.4	78.3	36m
aniso	RGB	early	no	yes	8	30.82	0.983	0.017	379	705	8.0	97.8	30m
aniso	RGB	early	yes	no	9	30.99	0.982	0.018	314	626	8.4	102.6	118m
aniso	RGB	early	yes	yes	8	30.83	0.982	0.018	594	690	8.6	105.3	102m
aniso	RGB	final	no	no	9	30.97	0.983	0.018	282	632	7.4	90.9	45m
aniso	RGB	final	no	yes	8	30.86	0.983	0.017	1474	678	8.0	97.5	31m
aniso	RGB	final	yes	no	8	30.94	0.982	0.019	304	621	6.7	82.2	109m
aniso	RGB	final	yes	yes	8	30.97	0.982	0.017	315	659	8.8	107.4	96m
aniso	RGB	none	no	no	9	30.90	0.983	0.018	395	632	7.5	91.7	48m
aniso	RGB	none	no	yes	8	30.82	0.983	0.017	425	680	8.1	99.2	32m
aniso	RGB	none	yes	no	9	31.03	0.983	0.018	1140	631	8.2	100.4	109m
aniso	RGB	none	yes	yes	8	31.01	0.983	0.017	531	674	8.8	107.6	92m
aniso	RGB	prune+dense	no	no	11	31.56	0.985	0.016	1005	620	6.2	75.4	52m
aniso	RGB	prune+dense	no	yes	10	31.35	0.984	0.017	709	669	7.5	91.3	48m
aniso	RGB	prune+dense	yes	no	13	31.51	0.985	0.017	359	365	9.5	117.0	169m
aniso	RGB	prune+dense	yes	yes	10	31.36	0.984	0.017	390	370	8.6	104.9	127m
aniso	4DSH	early	no	no	8	31.53	0.983	0.018	358	742	44.6	72.0	38m
aniso	4DSH	early	no	yes	8	31.57	0.984	0.015	452	846	55.2	89.2	37m
aniso	4DSH	early	yes	no	10	31.55	0.984	0.016	254	742	72.0	116.2	136m
aniso	4DSH	early	yes	yes	10	31.76	0.984	0.016	397	815	88.9	143.5	133m
aniso	4DSH	final	no	no	8	31.45	0.983	0.018	404	738	43.4	70.0	37m
aniso	4DSH	final	no	yes	8	31.36	0.983	0.017	1046	834	55.7	90.0	37m
aniso	4DSH	final	yes	no	10	31.56	0.984	0.016	410	797	71.5	115.4	157m
aniso	4DSH	final	yes	yes	10	31.55	0.984	0.016	432	1161	90.3	145.7	158m
aniso	4DSH	none	no	no	8	31.43	0.983	0.017	1387	739	43.7	70.4	36m
aniso	4DSH	none	no	yes	8	31.53	0.983	0.016	799	833	55.2	89.1	36m
aniso	4DSH	none	yes	no	10	31.77	0.984	0.016	413	765	71.5	115.4	131m
aniso	4DSH	none	yes	yes	8	31.66	0.984	0.017	508	735	62.4	100.8	103m
aniso	4DSH	prune+dense	no	no	13	31.83	0.984	0.016	422	710	50.1	80.8	57m
aniso	4DSH	prune+dense	no	yes	10	31.89	0.984	0.016	386	731	50.2	81.0	44m
aniso	4DSH	prune+dense	yes	no	10	31.84	0.984	0.020	398	587	49.0	79.1	142m
aniso	4DSH	prune+dense	yes	yes	10	32.08	0.985	0.017	200	1246	58.3	94.0	137m
iso	RGB	early	no	no	7	27.88	0.972	0.037	694	668	4.8	110.4	38m
iso	RGB	early	no	yes	6	27.79	0.971	0.039	940	720	3.4	78.0	23m
iso	RGB	early	yes	no	7	28.12	0.972	0.038	289	782	4.2	96.7	88m
iso	RGB	early	yes	yes	7	27.90	0.972	0.036	1421	647	6.1	141.1	76m
iso	RGB	final	no	no	6	27.81	0.971	0.042	1114	665	2.5	58.2	28m
iso	RGB	final	no	yes	6	27.77	0.971	0.038	1105	761	3.3	76.7	23m
iso	RGB	final	yes	no	8	28.05	0.973	0.034	305	615	5.6	129.6	100m
iso	RGB	final	yes	yes	7	27.99	0.972	0.035	570	638	6.1	140.2	78m
iso	RGB	none	no	no	7	27.84	0.972	0.036	453	635	4.7	107.7	34m
iso	RGB	none	no	yes	7	27.75	0.972	0.035	985	733	6.3	144.9	27m
iso	RGB	none	yes	no	7	28.10	0.972	0.038	728	581	4.1	94.2	80m
iso	RGB	none	yes	yes	6	27.95	0.972	0.041	402	671	3.3	76.7	67m
iso	RGB	prune+dense	no	no	11	28.07	0.974	0.031	354	626	7.5	173.3	55m
iso	RGB	prune+dense	no	yes	9	27.89	0.973	0.032	298	685	7.1	163.1	44m
iso	RGB	prune+dense	yes	no	11	28.14	0.974	0.030	474	455	7.4	170.2	143m
iso	RGB	prune+dense	yes	yes	8	28.13	0.973	0.033	475	794	5.0	115.0	90m
iso	4DSH	early	no	no	7	28.78	0.974	0.035	397	807	59.0	101.5	28m
iso	4DSH	early	no	yes	7	28.76	0.974	0.033	864	934	77.7	133.7	30m
iso	4DSH	early	yes	no	7	28.92	0.974	0.036	964	715	52.3	89.9	102m
iso	4DSH	early	yes	yes	7	28.91	0.974	0.034	423	820	74.6	128.4	102m
iso	4DSH	final	no	no	7	28.71	0.973	0.035	490	1785	57.4	98.7	29m
iso	4DSH	final	no	yes	7	28.79	0.974	0.033	206	942	77.0	132.4	29m
iso	4DSH	final	yes	no	7	28.84	0.974	0.035	211	1561	51.4	88.4	102m

ABLATIONS					VISUAL								
Isotropy	Color	Prune	Drop	ESS	Iter $\times 10^3$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	VRAM $\downarrow$	MB $\downarrow$	Gauss $\times 10^3 \downarrow$	Train m $\downarrow$
iso	4DSH	final	yes	yes	7	29.07	0.974	0.033	1366	2045	74.5	128.1	107m
iso	4DSH	none	no	no	7	28.79	0.973	0.035	567	1514	57.7	99.3	32m
iso	4DSH	none	no	yes	7	28.85	0.974	0.033	940	941	77.4	133.1	33m
iso	4DSH	none	yes	no	7	28.85	0.973	0.036	541	681	51.3	88.2	102m
iso	4DSH	none	yes	yes	7	28.95	0.974	0.034	707	796	75.0	129.1	101m
iso	4DSH	prune+dense	no	no	8	28.70	0.973	0.034	438	672	44.1	75.9	37m
iso	4DSH	prune+dense	no	yes	7	28.83	0.973	0.034	394	736	51.6	88.7	31m
iso	4DSH	prune+dense	yes	no	10	28.64	0.973	0.032	488	588	72.5	124.7	116m
iso	4DSH	prune+dense	yes	yes	8	28.91	0.974	0.033	415	710	58.7	101.0	98m

Table 4.trex

C.1 Ablation Trex Evaluation

Ablation	Overall effect	Better on	Worse on	Net SMD	Mean SMD
isotropy=anisotropic	mostly beneficial	LPIPS (+2.55), SSIM (+1.63), PSNR (+1.42)	none	1.87	1.87
dropout=no_dropout	mostly beneficial	training time (+2.01), render FPS (+0.69)	none	1.35	1.35
pruning=interleaved_prune_densify	mostly beneficial	eval VRAM (+0.75)	none	0.75	0.75
appearance=rgb	compactness trade-off	checkpoint size (+2.78), eval VRAM (+0.64)	PSNR (-1.15), SSIM (-0.83), LPIPS (-0.52)	0.18	1.19
appearance=4DSH cfg: 4DSH	quality trade-off	PSNR (+1.15), SSIM (+0.83), LPIPS (+0.52)	checkpoint size (-2.78), eval VRAM (-0.64)	-0.18	1.19
pruning=final_pruning	FPS-oriented trade-off	render FPS	eval VRAM (-0.57)	-0.57	0.57
dropout=dropout	mostly harmful	none	training time (-2.01), render FPS (-0.69)	-1.35	1.35
isotropy=isotropic	mostly harmful	none	LPIPS (-2.55), SSIM (-1.63), PSNR (-1.42)	-1.87	1.87

Strongest individual effects

Ablation	Effect	Comparison	Change
appearance=rgb	improves checkpoint size	14.4 vs 135.2	-89.3%, very large, SMD=+2.78; lower is better
isotropy=anisotropic	improves LPIPS	0.0218 vs 0.0416	-47.6%, very large, SMD=+2.55; lower is better
dropout=no_dropout	improves training time	2128.7 vs 5362.1	-60.3%, very large, SMD=+2.01; lower is better
isotropy=anisotropic	improves SSIM	0.9797 vs 0.9713	+0.9%, large, SMD=+1.63; higher is better
isotropy=anisotropic	improves PSNR	31.78 vs 29.66	+7.2%, large, SMD=+1.42; higher is better
appearance=4DSH cfg: 4DSH	improves PSNR	31.63 vs 29.81	+6.1%, large, SMD=+1.15; higher is better
appearance=4DSH cfg: 4DSH	improves SSIM	0.9780 vs 0.9729	+0.5%, moderate, SMD=+0.83; higher is better
pruning=interleaved_prune_densify	improves eval VRAM	575.0 vs 752.8	-23.6%, moderate, SMD=+0.75; lower is better
dropout=no_dropout	improves render FPS	791.4 vs 493.7	+60.3%, moderate, SMD=+0.69; higher is better
appearance=rgb	improves eval VRAM	631.9 vs 784.8	-19.5%, moderate, SMD=+0.64; lower is better
appearance=4DSH cfg: 4DSH	improves LPIPS	0.0285 vs 0.0349	-18.4%, moderate, SMD=+0.52; lower is better

For completeness, we include the command that we used to generate the ablations:

```
uv run batch_train.py \
configs/dnerf_ablation/trex.yaml \
configs/dnerf_ablation/bouncingballs.yaml \
--matrix-preset cartesian \
--axes isotropy,appearance,sorting,pruning,dropout,ess \
--output-root output/ablations_fixed_longer_newer \
--max-iters 30000 \
--set "save_iterations=[10000,20000,30000]" \
--set "test_iterations=[1000,2000,3000,4000,5000,6000,7000,8000,9000,10000, \
                        11000,12000,13000,14000,15000, \
                        16000,17000,18000,19000,20000, \
                        21000,22000,23000,24000,25000, \
                        26000,27000,28000,29000,30000]" \
--set use_usplat=False \
--set ess_max_splits=500000 \
--set ess_from_iter=750 \
--set ess_until_iter=22500 \
--set densification_interval=750 \
--set densify_from_iter=750 \
--set densify_until_iter=22500 \
--interleaved-prune-from-iter 1500 \
--interleaved-prune-until-iter 11250 \
--interleaved-prune-interval 750 \
--interleaved-densify-from-iter 375 \
--interleaved-densify-until-iter 11250 \
--interleaved-densification-interval 375 \
--no-quota-reservation \
--slurm-driver \
--repo-root "$PROJECT_ROOT" \
--runner uv \
--uv-binary uv \
--slurm-nodes 2 \
--slurm-gpus 4 \
--slurm-gpus-per-node 2 \
--slurm-tasks 4 \
--slurm-tasks-per-node 2 \
--slurm-total-cpus 32 \
--slurm-mem 300G \
--ablations-per-gpu 5
```

D USPLAT Ablations

ABLATIONS					VISUAL								
Color	Prune	Drop	ESS	USplat	Iter $\times 10^3$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	VRAM $\downarrow$	MB $\downarrow$	Gauss k $\downarrow$	Train m $\downarrow$
RGB	prune+dense	yes	yes	no	7	26.60	0.958	0.042	1649	53	6.3	25.3	1m
RGB	prune+dense	yes	yes	yes	7	26.72	0.958	0.039	1682	54	6.7	27.1	29m
RGB	prune+dense	yes	no	no	7	26.54	0.958	0.044	1692	53	5.8	23.6	1m
RGB	prune+dense	yes	no	yes	7	26.64	0.959	0.041	1718	53	6.2	25.0	27m
RGB	prune+dense	no	yes	no	7	27.57	0.962	0.035	1672	53	6.5	26.2	1m
RGB	prune+dense	no	yes	yes	7	27.51	0.960	0.037	1865	54	6.9	28.1	28m
RGB	prune+dense	no	no	no	7	27.41	0.961	0.036	1916	53	6.0	24.3	1m
RGB	prune+dense	no	no	yes	7	27.61	0.961	0.036	1901	53	6.2	25.0	26m
RGB	none	yes	yes	no	7	26.81	0.959	0.041	1632	53	6.3	25.7	1m
RGB	none	yes	yes	yes	7	26.68	0.958	0.042	1629	54	6.7	27.2	32m
RGB	none	yes	no	no	7	26.57	0.958	0.041	1648	53	5.9	23.9	1m
RGB	none	yes	no	yes	7	26.69	0.959	0.043	1661	53	6.3	25.6	20m
RGB	none	no	yes	no	7	27.52	0.961	0.037	1830	53	6.4	25.8	1m
RGB	none	no	yes	yes	7	27.46	0.961	0.039	1831	54	6.8	27.6	30m
RGB	none	no	no	no	7	27.54	0.962	0.037	1865	53	6.0	24.1	1m
RGB	none	no	no	yes	7	27.40	0.961	0.037	1630	53	6.3	25.4	19m
4DSH	prune+dense	yes	yes	no	7	28.98	0.965	0.035	1302	103	50.0	26.9	2m
4DSH	prune+dense	yes	yes	yes	7	28.82	0.963	0.035	1281	109	52.6	28.3	30m
4DSH	prune+dense	yes	no	no	7	28.77	0.963	0.038	1380	100	46.0	24.7	2m
4DSH	prune+dense	yes	no	yes	7	28.90	0.964	0.037	1370	101	48.1	25.8	28m
4DSH	prune+dense	no	yes	no	7	30.17	0.971	0.028	1448	100	47.7	25.6	1m
4DSH	prune+dense	no	yes	yes	7	30.19	0.971	0.028	1428	103	50.0	26.9	28m
4DSH	prune+dense	no	no	no	7	30.02	0.970	0.028	1535	96	44.2	23.8	1m
4DSH	prune+dense	no	no	yes	7	30.03	0.970	0.028	1543	100	45.6	24.5	26m
4DSH	none	yes	yes	no	7	28.91	0.965	0.034	1300	102	49.2	26.4	2m
4DSH	none	yes	yes	yes	7	28.88	0.964	0.036	1271	109	51.9	27.9	30m
4DSH	none	yes	no	no	7	28.81	0.964	0.039	1357	100	45.7	24.5	2m
4DSH	none	yes	no	yes	7	28.97	0.965	0.035	1334	101	48.9	26.3	21m
4DSH	none	no	yes	no	7	30.11	0.970	0.029	1449	100	46.6	25.1	1m
4DSH	none	no	yes	yes	7	30.16	0.971	0.027	1419	102	49.3	26.5	29m
4DSH	none	no	no	no	7	30.03	0.971	0.028	1415	94	43.5	23.4	1m
4DSH	none	no	no	yes	7	30.23	0.971	0.028	1512	100	45.3	24.3	19m

Table 5. Ablations: bouncingballs, 7k iter, sort, (200x200)

E MOG - Dataset



5 x moving Cameras



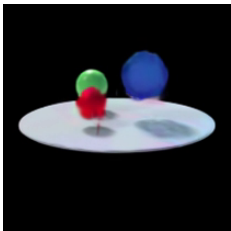
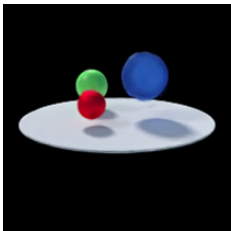
5 x still Cameras

We also recorded the “MOG” real-world dataset, which consists of 5 videos captured with handheld mobile phones from different angles. The dataset depicts a seated subject, who raises his eyebrows, removes his glasses and puts them back on. We synchronize the videos using clapping as auditory signals to identify beginning and end of the clip. The cameras rotate in swirling motions while fixated on the subject, for all-round imaging with fewer floating artifacts. We use MOG only as a qualitative capture, not for quantitative comparison.

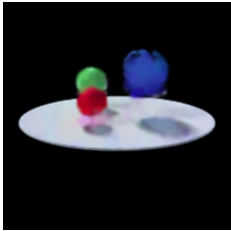
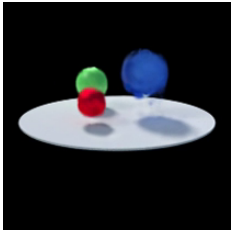
F General Comparison

Fixed: No USplat/Prune/ESS/Dropout, Sort, 10k.
Sort Sort-Free

Ellipsoid

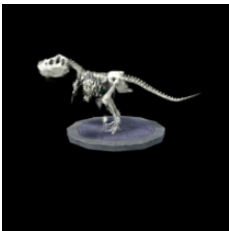


Spherical



Fixed: No USplat/Prune/ESS/Dropout, Sort, 20k.
4DSH RGB

Ellipsoid



Spherical

